M

Software engineering

Authors

Matt ter Steege

Matttersteege@gmail.com

October 22, 2026

Contents

1 Software requirements . . . 2
1.1 Wat zijn requirements? . . . 2
1.2 Hoe schrijf je requirements . . . 2
1.3 Requirements Engineering . . . 2
1.4 Use Cases . . . 3
1.5 User Stories . . . 3
1.6 Quick Prototype . . . 3
1.7 A/B Testing . . . 3
2 UML . . . 4
2.1 Relations . . . 4
3 Design patterns . . . 5
3.1 Strategy . . . 5
3.2 Adapter . . . 5
3.3 Facade . . . 6
3.4 Bridge . . . 6
3.5 Decorator . . . 6
3.6 Factory Method . . . 6
3.7 Observer . . . 7
3.8 Singleton . . . 7
3.9 Command . . . 7
3.10 Builder . . . 7
3.11 Template . . . 8
3.12 SOLID principepe . . . 8
4 Processen . . . 8
4.1 SWEBOK . . . 8
4.2 Waterfall . . . 9
4.3 Agile . . . 9
5 Software testing . . . 9
5.1 Unit testing . . . 9
5.2 Writing tests . . . 10
5.3 Problems with unit testing . . . 10
5.4 Andere test soorten . . . 10
6 Software architecture . . . 10

1 | Software requirements

Wat zijn requirements?

"Requirements define what a system should do and under what constraints it should operate"

Als je een systeem ontwikkeld, dan hebben de gebruikers vaak eisen waaraan je wilt voldoen. Requirements engineering bevorderd de communicatie tussen stakeholders[1] en geeft het eindproduct dus meer sturing. Het kost wel geld...

Requirements moeten: **Correct, Precise, Complete, Consistant & Verifiable** zijn (alternatief is SMART). Er zijn 2 soorten requirements:

Functionele requirements

Wat moet een systeem doen

Non-Functionele requirements

Onder welke condities, vaak met metrieken. VOorbeelden hiervan zijn bijvoorbeeld transacties per seconden, laadtijd, opslaggebruik, RAM gebruik, uptime percentage etc. dingen die wel te meten zijn, maar niet functioneel zijn.

Hoe schrijf je requirements

Requirements moeten aan een aantal eigenschappen voldoen:
- **Correct** – beschrijft de juiste operaties en berekeningen.
- **Precise** – er is geen ambiguïteit in de beschrijving.
- **Complete** – alle noodzakelijke functionaliteiten zijn beschreven.
- **Consistent** – alle stakeholders hebben dezelfde interpretatie van de specificatie.
- **Verifiable** – het is mogelijk om te controleren of de implementatie aan de requirement voldoet.

Een alternatief hiervoor is SMART: Specific, Measurable, Achievable, Relevant, Time-bound.

Voorbeeld **(slechte requirements)**:

"The system should be usable by elderly people." Dit is niet goed verifieerbaar en "elderly" is onduidelijk.
"The product shall respond rapidly." "Rapidly" is niet concreet en dus niet verifieerbaar.
Beter: "The system shall respond in less than 2s in 99% of the cases."

Requirements Engineering

Requirements Engineering is het identificeren, analyseren, specificeren en valideren van de requirements van een systeem. Dit gebeurt in overleg tussen developers en stakeholders. Een stakeholder is een persoon, groep of entiteit die een belang heeft bij of zich zorgen maakt over de realisatie van het systeem.

Requirements verzamelen

Requirements kunnen op verschillende manieren worden verzameld:
- Praten met stakeholders (interviewsm brainstorms, vragenlijsten)
- Bestaande documenten en systemen analyseren.
- Observeren hoe toekomstige gebruikers momenteel werken.
- Prototyping.
- Task analysis: onderzoeken welke subtaken er zijn en hoe deze in een hiërarchie kunnen worden georganiseerd.
- Scenario-based analysis: onderzoeken hoe een taak wordt uitgevoerd en wat er gebeurt in uitzonderingssituaties.

[1] Een persoon, groep of entiteit die te maken heeft met het maken van het product

Use Cases

Een Use Case is een lijst van acties of gebeurtenissen die de interactie beschrijft tussen een rol (*actor*) en een systeem om een bepaald doel te bereiken. De actor kan een mens of een ander extern systeem zijn. Use Cases zijn vooral geschikt voor het identificeren van **functionele requirements op hoog niveau**. Ze worden beschreven in gestructureerde tekst en zijn gedetailleerder dan user stories.

User Stories

User stories zijn een lightweight aanpak voor requirements en use-case-analyse. Ze beschrijven hoe een applicatie naar verwachting gebruikt wordt en helpen stakeholders bij het plannen en prioriteren van de ontwikkeling. Een user story bevat drie onderdelen, ook wel de 3 R's genoemd:
- **Role** – wie wil iets?
- **Requirement** – wat wil die persoon?
- **Reason** – waarom wil die persoon dit?

De standaardvorm is: "As a *[role]*, I want *[requirement]*, so that *[reason]*."

Voorbeeld **(User story)**:

> As a new user, I want to sign up using my existing Google account so that I don't have to keep track of another account.

Een functionele requirement:

> The system must allow users to search for songs, albums, and artists using keywords.

kan bijvoorbeeld worden herschreven als:

> As a user, I want to search for songs, albums, and artists using keywords, so that I can easily find and listen to the music I enjoy.

Quick Prototype

Een andere manier om requirements te ontwikkelen is een **Quick Prototype**. Hierbij werk je nauw en continu samen met stakeholders om requirements en tests te ontwikkelen. Er wordt een werkend prototype onderhouden terwijl nieuwe features worden toegevoegd. Dit gebeurt typisch in korte iteraties, bijvoorbeeld iedere twee weken. Na iedere iteratie wordt met stakeholders besproken wat er als volgende moet worden toegevoegd. Hierdoor kan worden gecontroleerd of het huidige systeem daadwerkelijk overeenkomt met wat stakeholders willen.

Minimum Viable Product (MVP)

Een Minimum Viable Product (MVP) is een subset van de volledige verzameling features die, wanneer deze af is, al *business value in de echte wereld* heeft. Het doel is dus niet om meteen het volledige systeem te bouwen, maar om een minimale versie te maken die al waarde heeft.

A/B Testing

Bij A/B testing worden minimaal twee versies van een systeem aan verschillende groepen gebruikers getoond. Elke groep krijgt een andere versie te zien. Bijvoorbeeld:
- Versie A implementeert requirement A.
- Versie B implementeert requirement B.

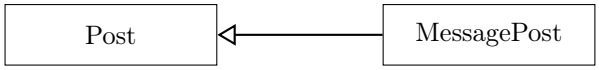
De requirements zijn hierbij onderling exclusief. Door de verschillende versies aan gebruikers te tonen, kan worden onderzocht welke variant het beste aansluit bij het gewenste gebruik.

2 | UML

Relations

Inheritance / Generalization

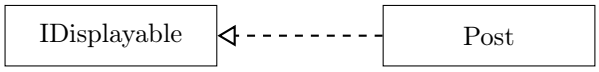
Inheritance means that one class is a specialized version of another: an **is-a** relationship.



Lees dit als: **MessagePost is a Post**
De lege driehoek wijst naar de superclass, dus Subclass ⟶ Superclass. Het voorbeeld uit het hoorcollege is dat "`MessagePost` is derived from `Post`".

Realization

Realization is used when a class **implements an interface**.



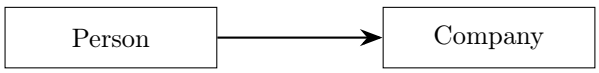
Lees dit als: **Post implements IDisplayable**
het verschil met Inheritance is:

solid + empty triangle	= inheritance
dashed + empty triangle	= realization

De driehoek wijst naar de interface die geimplementeerd wordt.

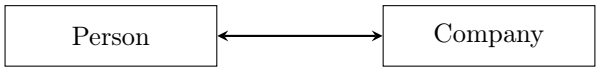
Association

Een associatie houdt in dat klassen met elkaar verbonden zijn, doorgaans doordat de ene klasse **weet van** de andere.



Lees dit als: **Person knows Company**
De pijl wijst naar de class waarover geweten wordt (lekker genederlandst). Knower ⟶ Known class.

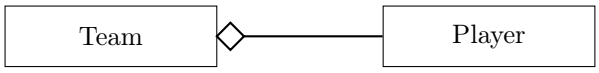
Een pijl die beide richtingen opgaat (bidirectionaal) betekend dat beide klassen van elkaar weten:



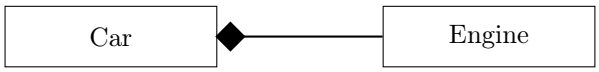
Het voorbeeld was dat een persoon weet bij welk bedrijf hij werkt, en het bedrijf weet dat hij bij hun werkt.

Aggregation and Composition

Beide beschrijven een **part-of** / **has-a**-relatie. Het verschil zit in de sterkte van het eigendom.



Aggragation gebruikt een lege diamant. De diamant is aan de kan van de "container" kant.
Lees dit als: **Team has Players**
Het belangrijke is dat een deel kan bestaan zonder de ander, een team bestaat ook wel zonder een bepaalde speler, en vice versa.

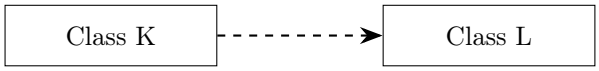


Composition gebruikt een gevulde diamant. De diamant is aan de kan van de "container" kant. Compositie is een **sterke vorm van aggregatie**. Het ingesloten object maakt deel uit van het omvattende object en heeft geen zelfstandig bestaansrecht.
Lees dit als: **Car requires engine & engine requires a car**

Voor het examen: Diamond always goes on the "whole" side.

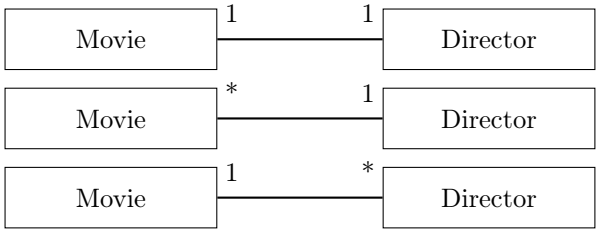
Dependency

Een dependency houdt in dat de ene klasse een andere klasse **gebruikt of daarvan afhankelijk is**. Het college noemt ook het creëren van een andere klasse als voorbeeld.



Lees dit als: **K depends on L**
De pijl wijst naar de class die gebruikt wordt. Een belangrijk onderscheid is dat een afhankelijkheid **niet** noodzakelijkerwijs betekent dat K een L-object als attribuut opslaat. Meer in het algemeen geldt: als een wijziging in L gevolgen kan hebben voor K, dan is K afhankelijk van L.

Multiplicity / Cardinality



Multiplicity geeft aan **hoeveel objecten** aan een relatie kunnen deelnemen. Veer voorkomende voorbeelden zijn:

`1`	exactly one
`0..1`	zero or one
`*`	zero or many
`1..*`	one or many

Het voorbeeld hiernaast zegt:
- Elke film heeft precies één regisseur.
- Elke regisseur heeft 0 of meer films geregisseerd.
- Elke film kan meerdere regisseurs hebben.

3 | Design patterns

Strategy

Intent: verschillende algoritmen kunnen gebruiken afhankelijk van de context.
Probleem: de client moet zelf kiezen en omgaan met verschillende algoritmen.
Oplossing: definieer een gemeenschappelijke interface en laat verschillende strategies deze implementeren.
Code voorbeeld:

```
interface ISort { void Sort(int[] a); }
class QuickSort : ISort { public void Sort(int[] a) {} }
class App {
    ISort sort;
    void Run(int[] a) => sort.Sort(a);
}
```

Adapter

Intent: een bestaand object passend maken bij een gewenste interface.
Probleem: de bestaande klasse bevat de juiste functionaliteit, maar heeft de verkeerde interface.
Oplossing: maak een adapter die de bestaande klasse omhult en de gewenste interface aanbiedt.
Code voorbeeld:

```
interface IPrinter { void Print(); }
class OldPrinter { public void OldPrint() {} }
class Adapter : IPrinter {
    OldPrinter p = new();
    public void Print() => p.OldPrint();
}
```

Facade

Intent: het gebruik van een complex systeem vereenvoudigen.
Probleem: de client moet met veel onderliggende klassen communiceren.
Oplossing: maak één eenvoudige interface die de complexiteit verbergt.
Code voorbeeld:

```
class CPU { public void Start() {} }
class Disk { public void Load() {} }
class Computer {
    CPU cpu = new(); Disk disk = new();
    public void Start() { cpu.Start(); disk.Load(); }
}
```

Bridge

Intent: abstractie en implementatie van elkaar loskoppelen.
Probleem: combinaties van abstracties en implementaties veroorzaken veel klassen.
Oplossing: laat de abstractie een aparte implementatie-interface gebruiken via compositie.
Code voorbeeld:

```
interface IDevice { void On(); }
class TV : IDevice { public void On() {} }
class Remote {
    IDevice device;
    public Remote(IDevice d) => device = d;
    public void On() => device.On();
}
```

Decorator

Intent: gedrag van een object dynamisch uitbreiden.
Probleem: veel subklassen zijn nodig voor verschillende combinaties van extra gedrag.
Oplossing: wikkel een object in een decorator die hetzelfde interface gebruikt en extra gedrag toevoegt.
Code voorbeeld:

```
interface ICoffee { void Drink(); }
class Coffee : ICoffee { public void Drink() {} }
class Milk : ICoffee {
    ICoffee c;
    public Milk(ICoffee c) => this.c = c;
    public void Drink() { c.Drink(); /* + melk */ }
}
```

Factory Method

Intent: objectcreatie uitstellen naar subklassen.
Probleem: een klasse moet objecten maken zonder afhankelijk te zijn van een concrete klasse.
Oplossing: definieer een methode voor het maken van een object en laat subclasses bepalen welk object wordt gemaakt.
Code voorbeeld:

```
interface IProduct {}
class A : IProduct {}
abstract class Factory { public abstract IProduct Create(); }
class AFactory : Factory {
    public override IProduct Create() => new A();
}
```

Observer

Intent: meerdere objecten automatisch op de hoogte brengen van veranderingen.
Probleem: objecten moeten reageren wanneer de toestand van een ander object verandert.
Oplossing: laat observers zich registreren bij een subject dat hen bij veranderingen informeert.
Code voorbeeld:

```
interface IObserver { void Update(); }
class Subject {
    List<IObserver> obs = new();
    public void Change() => obs.ForEach(o => o.Update());
}
```

Singleton

Intent: slechts één instantie van een klasse beschikbaar maken.
Probleem: meerdere instanties van een object mogen niet bestaan.
Oplossing: maak de constructor private en geef één gedeelde instantie terug.
Code voorbeeld:

```
class Config {
    private static readonly Config instance = new();
    private Config() {}
    public static Config Instance => instance;
}
```

Command

Intent: een actie als object voorstellen.
Probleem: acties moeten bijvoorbeeld opgeslagen, uitgesteld of ongedaan gemaakt kunnen worden.
Oplossing: stop de actie in een Command-object met een gemeenschappelijke interface.
Code voorbeeld:

```
interface ICommand { void Execute(); }
class Light { public void On() {} }
class OnCommand : ICommand {
    Light l;
    public OnCommand(Light l) => this.l = l;
    public void Execute() => l.On();
}
```

Builder

Intent: een object maken dat vervolgens niet meer aangepastkan worde én aan verschillende eisen voldoet.
Probleem: Een complex object kan je op verschillende manieren maken, wat kan leiden tot lange constructors.
Oplossing: Isoleer het creeëren en splits het op in verschillende stappen/methoden.
Code voorbeeld:

```
public class OnlineCourse {
    public string CourseName { get; private set; } //meerdere
    private OnlineCourse() { } //private constructor

 public class Builder {
  private OnlineCourse _oc = new();
  public Builder SetCourseName(string a) {...} //meerdere set methodes
     public OnlineCourse Build() {
        //check hier of de variabelen correct zijn
  }
 }
}
```

Template

Intent: Defineer een skelet waar *sommige* taken door de subclass wordt gemaakt.
Probleem: Een algoritme is consisten op een bepaald level, maar heeft enkele object-specifieke implementaties.
Oplossing: Creeer subclassen voor een deel van het process en laat de rest hetzelfde.
Code voorbeeld:

```
public abstract class Payment {
    public void ProcessPayment() {step1(); step2(); step3() /*...*/}

    protected abstract void step1(); //wordt overschreven door subclass
    private abstract void step2(); //blijven hetzelfde onafhankelijk van het algoritme
    private abstract void step3();
}
```

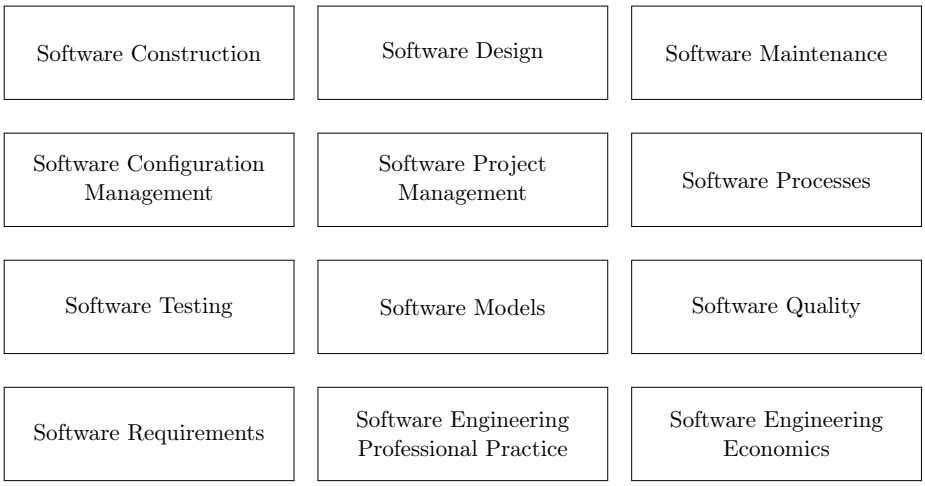
SOLID principepe

De **SOLID**-principes zijn vijf basisregels in software development die zorgen voor begrijpelijke, flexibele en onderhoudbare code.
- **S - Single Responsibility Principle (SRP):** Een klasse mag maar één verantwoordelijkheid of reden tot verandering hebben. Dit houdt klassen klein en gefocust.
- **O - Open/Closed Principle (OCP):** Software-entiteiten (klassen, modules, functies) moeten open zijn voor uitbreiding, maar gesloten voor aanpassing. Je voegt nieuwe functionaliteit toe zonder bestaande code te herschrijven.
- **L - Liskov Substitution Principle (LSP):** Subklassen moeten altijd uitwisselbaar zijn met hun basisklasse zonder dat het gedrag van het programma verandert. Er mag geen onverwacht gedrag optreden bij overerving.
- **I - Interface Segregation Principle (ISP):** Klassen mogen niet gedwongen worden om interfaces te implementeren die ze niet gebruiken. Het is beter om meerdere specifieke interfaces te hebben dan één grote 'all-in-one' interface.
- **D - Dependency Inversion Principle (DIP):** Afhankelijkheden moeten liggen bij abstracties (interfaces), niet bij concrete implementaties. High-level modules mogen niet direct afhankelijk zijn van low-level modules.

4 | Processen

Een serie stappen, taken of practises die je volgt als je software ontwikkeld, dit doe je in een team. Processen zorgen voor kwaliteit, productiviteit, alignment met de doelen van de organisatie.

SWEBOK



Waterfall

De watervalmethode is een methode voor softwareontwikkeling, waarin de ontwikkeling regelmatig vloeiend naar beneden loopt. De ontwikkeling loopt door een aantal fasen, namelijk:
1. Analyse en eisen
2. Ontwerp
3. Bouw/realisatie
4. Testen, hier ga je dus unit test schrijven
5. Opleveren en onderhoud

Voordelen zijn:
- Gestructureerd
- Duidelijke fases
- Focus op documentatie
- Testen (als er tijd is)

Nadelen zijn:
- Requirements die vaak veranderen
- De klant wordt pas laat betrokken
- Duurt erg lang om software te ontwikkelen
- Resulteert in dure projecten

Agile

Agile process zijn gebaseerd op increments/sprints, die ongeveer 1/4 weken duren. Systemen worden gebouwd door kleine delen toe te voegen. Je maakt een Minimal Viable Product (MVP). Dat lever je op en daarna vraag je feedback van de klant/gebruiker. Elke sprint heeft ook zijn eigen test periode (geautomatiseerd). Agile is dus ook wel "Test-driven development" (TDD), je start met een iteratie van tests schrijven, daarna maak je code waardoor de tests slagen, als laatste refactor je de solution.

Voordelen zijn:
- Klanten worden beter betrokken
- Adaptief
- Minder risico

Nadelen zijn:
- Lastig om te implementeren
- Moeilijk te schalen
- gefragmenteerde features

Scrum

Dit is de meest voorkomende agile methode, het is niet specifiek alleen voor software engineering. Het is interatief, opbouwend en simpel. Scrum is redelijk gestructureerd, het bestaat uit verschillende rollen, artefacten en events. Het is heel transparant omdat je kan zien hoe ver iedereen is. Er zijn tussen de 8-10 mensen in een scrum team, hier is één product owner en één scrum master, de rest zijn developers. Iedereen moet alles kunnen, het is cross-functional.
- **Product backlog**: Een geprioriteerde lijst van user stories
- **Sprint backlog**: Een bepaalde set die deze sprint af moet zijn.
- **Increment**: Concrete stap naar het product goal
- **Standup**: Dagelijkse korte update over de voortgang en eventuele obstakels
- **Product goal**: Het eindresultaat
- **Sprint goal**: " van de sprint
- **Sprint review**: Evaluatie van het opgeleverde werk met stakeholders
- **Sprint planning**: Overleg waarin het doel en werk voor de nieuwe sprint worden bepaald
- **Retrospective**: Evaluatie van het teamproces om de samenwerking te verbeteren

5 | Software testing

Unit testing

Unit testing verifieert wat een methode terug geeft aan de hand van verschillende variabelen. Tests moeten **FIRST** zijn: Fast, Isolated/Independent, Repeatable, Self-Checking, Timely.

Unit tests zijn voor de code, niet voor de GUI, en je test ook geen code die niet breekt (getters and setters). Dit is een van de smells van unit testing, dit zijn alle 3/19 smells van https://testsmells.org:
- **Assertion Roulette:** Meerdere assertions zonder duidelijke foutmelding.

- **Conditional Test Logic:** Test bevat conditionele of herhalende controlestructuren.
- **Redundant Print:** Test bevat overbodige printstatements.

Writing tests

Test coverage

Dit is het deel van de code die daadwerkelijk getest wordt, dus stel dat je deze code hebt:

```
...
if (state == true)
    throw new Exception();
state = false;
...
```

En de tests die je bedacht hebt zorgen nooit voor een codepath waar state false is, dan wordt de exception niet getest en zorgt dat dus voor lagere test coverage. Het is dus ook een percentage van code die uitgevoerd wordt door de tests. Hoger is vaak beter, want dan is de kans op onvoorziene bugs lager:

$$\text{Test coverage} = \frac{\text{Statements uitgevoerd door tests}}{\text{Totaal aantal statements in de code}} * 100\%$$

Echter is 100% test coverage onnodige en zelfs contraproductief.

Ook moet je zorgen dat de tests aan de AAA (Arrange/Act/Assert) voldoen:
- Arrange: Set up the test environment, initialize objects, create variables, seed data, and configure mocks.
- Act: Execute the target method, function, or behavior being tested, and capture any resulting output.
- Assert: Verify that the actual outcome matches the expected result using framework assertion statements.

Problems with unit testing

tests functies die een collaborator nodig hebben kunnen problemen veroorzaken. Zoals een test van een complex systeem dat met een database interacteerd. Hiervoor kan je test doubles gebruiken: “namaak objecten die je gebruikt in plaats van echte objecten.” Voorbeelden hiervan zijn mocks of stubs. Dit zijn stukjes code die de echte methode vervangen door vast gedrag voor de tests.

Andere test soorten

- **Property-based testing**: In plaats van input/output (in: 3, geeft uit: 4) geef je invarianten waaraan je functie moet voldoen, dus output is input + 1, oid.
- **Integration tests**: Dit test of een compisitie van unit tests correct werken, dit kan dus ook UI zijn.
- **System testing**: Dit test een heel systeem, ook de use cases/user stories, en de NFR’s (vaak door QA-team)
- **Acceptance testing**: Hier test je of de user/client het uiteindelijke product accepteert.
- **Regression testing**: Verifeer dat voorgaande functionaliteit nogsteeds werkt.

6 | Software architecture